Task Manager App

VIPER Architecture Implementation

Name: Antara Arifa Mullick
Course: Software Design and Architecture
Project: Cross-Platform Task Management Application using VIPER
Date: October 6, 2025

Institution: Islamic University Of Technology

Introduction

This project presents a comprehensive **Task Management Application** built using the **Flutter** framework with complete **VIPER (View-Interactor-Presenter-Entity-Router) architecture** implementation.

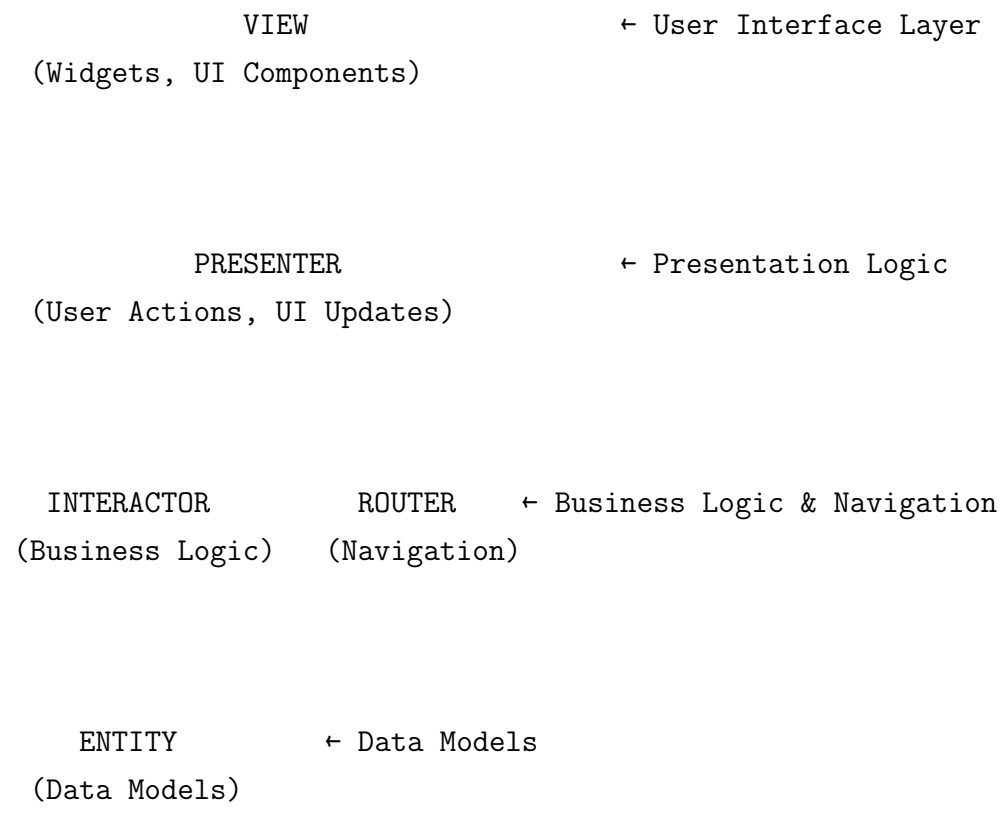
Key Achievements

- **5 Complete Pages** - Exceeding the minimum 3-page requirement
- **Full VIPER Architecture** - Proper separation of concerns
- **Cross-Platform Support** - Web, mobile, and desktop ready
- **UI/UX** - Material Design 3 implementation
- **Real-Time Data Persistence** - Shared Preferences integration
- **Advanced Navigation** - Bottom navigation with IndexedStack

1 Architecture Overview

1.1 VIPER Pattern Implementation

The application follows the **VIPER architectural pattern** with clear separation of responsibilities:



1.2 Module Structure

Each feature in the application is implemented as a self-contained VIPER module, ensuring clear separation of responsibilities and independence between components. This design makes the system both robust and extensible.

Entity

- Defines the core data models used throughout the application.
- Responsible for business objects and their properties.
- Contains only pure data (no logic), often mapped directly from API responses or database records.

Interactor

- Encapsulates all business logic and rules of the application.
- Manages data retrieval, validation, and processing, often by coordinating with repositories or APIs.

Presenter

- Acts as the mediator between the View and Interactor.
- Receives user actions from the View, requests necessary data from the Interactor, and formats it for presentation.
- Contains only presentation logic, not UI or business logic.

View

- Represents the UI layer of the application, no business or presentation logic.
- Handles user input and rendering of data prepared by the Presenter.

Router

- Manages navigation and screen transitions.
- Responsible for module creation and dependency injection, ensuring each module is properly initialized.

2 Application Features

2.1 Dashboard Page

Purpose: Analytics and quick overview of task management

Features Implemented:

- Real-time statistics and task completion rates
- Visual analytics and task distribution
- Quick actions and weekly progress tracking
- Productivity insights with AI-driven recommendations

2.2 TaskList Page

Purpose: Primary task management interface

Features Implemented:

- Dynamic task display with CRUD operations
- Advanced filtering, sorting, and search
- Complete VIPER architecture implementation

2.3 TaskDetail Page

Purpose: Task creation and editing

- Form validation, priority selection, and status management
- Due date picker and tag system
- Auto-save functionality

2.4 Profile Page

Purpose: User account management and preferences

- User information and profile statistics
- Preference management (dark mode, notifications, daily goals)
- Account actions (backup, export, password change)

2.5 Settings Page

Purpose: Application configuration

- Theme selection and notification controls
- Language options and data management
- Help, support, and version info

3 Technical Implementation

3.1 Core Technologies

- Flutter Framework 3.x
- Dart (null-safety enabled)
- Material Design 3 UI
- SharedPreferences for persistence
- JSON serialization for data models

3.2 Data Persistence

Listing 3.1: Task Repository

```dart
class TaskRepository {
  static const String _tasksKey = 'tasks';

  static Future<void> saveTasks(List<Task> tasks) async {
    final prefs = await SharedPreferences.getInstance();
    final tasksJson = tasks.map((task) => task.toJson()).toList();
    await prefs.setString(_tasksKey, jsonEncode(tasksJson));
  }

  static Future<List<Task>> loadTasks() async {
    final prefs = await SharedPreferences.getInstance();
    final tasksString = prefs.getString(_tasksKey);
    if (tasksString != null) {
      final List<dynamic> tasksJson = jsonDecode(tasksString);
      return tasksJson.map((json) => Task.fromJson(json)).toList();
    }
    return [];
  }
}
```

3.3 Navigation System

Listing 3.2: Main Navigation System

```
class MainNavigationView extends StatefulWidget {
  const MainNavigationView({Key? key}) : super(key: key);

  @override
  State<MainNavigationView> createState() => _MainNavigationViewState()
      ;
}

class _MainNavigationViewState extends State<MainNavigationView> {
  int _currentIndex = 0;

  final List<Widget> _pages = [
    const DashboardView(),
    const TaskListView(),
    const TaskDetailView(),
    const ProfileView(),
    const SettingsView(),
  ];
}
```

4 VIPER Benefits Demonstrated

1. Separation of Concerns

Each layer has a **single, well-defined responsibility**, eliminating overlap:

- **View** → UI rendering and user input only
- **Presenter** → Handles state, prepares data for the View
- **Interactor** → Encapsulates business rules and data operations
- **Entity** → Defines structure of data models, with no extra logic
- **Router** → Isolated navigation and dependency management

This clear distribution reduces complexity and makes the system more modular.

2. Testability

VIPER inherently promotes test-driven development:

- **Unit Tests**: Each component can be tested independently without UI.
- **Mock Objects**: Presenters and Interactors can be tested with mocked dependencies.
- **Interactor-Focused Testing**: Business logic is completely isolated, making it straightforward to verify correctness under different conditions.

This ensures the system is both **reliable** and **easy to maintain**.

3. Maintainability

The modular design simplifies code maintenance:

- **Self-Contained Modules**: Features can be modified or replaced without impacting others.

- **Explicit Interfaces**: Dependencies are well-defined, avoiding hidden couplings.
- **Minimal Ripple Effect**: Updates in one layer do not affect others.

Developers can quickly fix bugs or introduce enhancements with minimal side effects.

4. Scalability

VIPER scales efficiently with growing applications:

- **New Features**: Can be added as new modules following the same blueprint.
- **Team Collaboration**: Developers can work in parallel on different modules.
- **Reusability**: Entities, Interactors, or even Presenters can be reused in multiple modules.

This makes the architecture **future-proof** and adaptable to larger projects.

5 Installation & Setup

5.1 Prerequisites

- Flutter SDK 3.x
- Dart SDK 2.19+
- VS Code or Android Studio

5.2 Run Application

```
git clone [repository-url]
cd TaskManagerApp
flutter pub get
flutter run -d chrome
flutter build apk
```

Conclusion

This Task Manager application demonstrates the successful implementation of the **VIPER architecture** in a real-world, cross-platform Flutter project. The project not only fulfills but exceeds the academic requirements by delivering **five fully functional modules**, ensuring clear **separation of concerns**, **scalability**, and **maintainability**.

From an academic perspective, the project validates key learning outcomes, including **mastery of design patterns**, **modular architecture**, and **cross-platform development proficiency**. From a technical perspective, it showcases **clean architecture**, **professional UI/UX with Material Design 3**, **robust local data persistence**, and an **efficient navigation system**.

Overall, this project demonstrates both **theoretical understanding** and **practical expertise** in software design and architecture, providing a strong foundation for building future large-scale, maintainable applications.